Contents

1 Log setups

1.1 Namespace log `N1` - Simple namespace creation and deletion

- Create a namespace:

```
kubectl create namespace rising
```

- Get the namespace:

```
kubectl get namespaces
```

- Describe the namespace:

```
kubectl describe namespace rising
```

- Delete the namespace:

```
kubectl delete namespace rising
```

1.2 Namespace log `N2` - Namespace with resource quotas

- **Prerequisite**: recreate the `rising` namespace:

```
kubectl create namespace rising
```

- Create a ResourceQuota for the namespace:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: ResourceQuota
    metadata:
      name: rising-quota
    spec:
      hard:
        pods: "1"
        requests.cpu: "1"
        requests.memory: 1Gi
        limits.cpu: "2"
        limits.memory: 2Gi
    EOF
```

- Inspect the ResourceQuota:

```
kubectl -n rising describe resourcequota rising-quota
```

- Create a Pod that will exceed the quota:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: over-quota-pod
      namespace: rising
    spec:
      containers:
      - name: over-quota-container
        image: nginx
        resources:
          requests:
            cpu: "1"
            memory: 1.5Gi
          limits:
            cpu: "2"
            memory: 2Gi
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Now create a Pod that will respect the quota:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: within-quota-pod
      namespace: rising
    spec:
      containers:
      - name: within-quota-container
        image: nginx
        resources:
          requests:
            cpu: "1"
            memory: 1Gi
          limits:
            cpu: "2"
            memory: 2Gi
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Run a further Pod, it will fail:

```
kubectl -n rising run --image=nginx nginx
```

- Clean everything up, one pod will fail because it was never created:

```
kubectl -n rising delete pod over-quota-pod
kubectl -n rising delete pod within-quota-pod
kubectl delete resourcequota rising-quota
```

1.3 Pod log `P1` - Simple Pod creation and inspection

- Create a pod:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: rising-pod
      namespace: rising
    spec:
      containers:
      - name: first-container
        image: nginx
      - name: second-container
        image: alpine
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod rising-pod
```

- Patch the Pod and try to add a third container, this will fail:

```
kubectl -n rising patch pod rising-pod --type='json' -p='[{"op": "add", "path":
↪  "/spec/containers/1", "value": {"name": "third-container", "image": "nginx"}}]'
```

- Get the pod log:

```
kubectl -n rising logs rising-pod -c second-container
```

- Execute commands in the second containers, the first will complain about the shell, the second will work:

```
kubectl -n rising exec -t rising-pod -c second-container -- /bin/bash -c "echo Hello again,
↪  Kubernetes!"
kubectl -n rising exec -t rising-pod -c second-container -- /bin/sh -c "echo Hello again,
↪  Kubernetes!"
```

- Delete the pod:

```
kubectl -n rising delete pod rising-pod
```

1.4 Pod log `P2` - Image download

- **Prerequisite**: run the following helper script to wipe the image cache on all nodes:

```
for node in $(kubectl get nodes -o jsonpath='{.items[*].metadata.name}'); do
  echo "Pruning images on $node"
  ssh -F ssh/cluster-setup $node "sudo crictl rmi --prune"
done
```

- Create a Pod with a container that uses a non-existent image:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: image-pod
      namespace: rising
    spec:
      containers:
      - name: image-container
        image: www.fbk.eu/non-existent-image
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Get the pod log:

```
kubectl -n rising logs image-pod -c image-container
```

- Get the events that led to the failure:

```
kubectl get events -n rising --sort-by='.metadata.creationTimestamp' --selector
↪  'involvedObject.name=image-pod'
```

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

- Create a pod with an image that will surely not be in cache:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: image-pod
      namespace: rising
    spec:
      containers:
      - name: image-container
        image: busybox
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Wait for the Pod to be downloaded and monitor the events:

```
kubectl wait --for=condition=Ready pod/image-pod -n rising
kubectl get events -n rising --sort-by='.metadata.creationTimestamp'
```

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

1.5 ConfigMap `C1` - Create a ConfigMap and use it in a Pod, then delete it

- Create a ConfigMap:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: ConfigMap
    metadata:
      name: my-config
      namespace: rising
    data:
      status: "decent"
    EOF
```

- Get the configmap

```
kubectl -n rising get configmap my-config -o yaml
```

- Deploy a Pod that uses the ConfigMap:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: config-pod
      namespace: rising
    spec:
      containers:
      - name: config-container
        image: alpine
        command: ["sh", "-c", 'echo Status is \$STATUS && sleep 3600']
        env:
        - name: STATUS
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: status
    EOF
```

- Access the logs of the Pod:

```
kubectl wait --for=condition=Ready pod/config-pod -n rising
kubectl -n rising logs config-pod -c config-container
```

- Clean everything up:

```
kubectl -n rising delete pod config-pod
kubectl -n rising delete configmap my-config
```

1.6 Secret `S1` - Create a Secret and use it in a Pod, then delete it

- Create a secret

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Secret
    metadata:
      name: my-secret
      namespace: rising
    type: Opaque
    data:
      username: $(echo -n "admin" | base64)
      password: $(echo -n "password" | base64)
    EOF
```

- Inspect the secret:

```
kubectl -n rising get secret my-secret -o yaml
```

- Attach the secret to a Pod:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: secret-pod
      namespace: rising
    spec:
      containers:
      - name: secret-container
        image: alpine
        command: ["sh", "-c", 'echo Username is \$USERNAME && echo Password is \$PASSWORD &&
    sleep 3600']
        env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username
        - name: PASSWORD
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: password
    EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod secret-pod
```

- Access the logs of the Pod:

```
kubectl wait --for=condition=Ready pod/secret-pod -n rising
kubectl -n rising logs secret-pod -c secret-container
```

- Clean everything up:

```
kubectl -n rising delete pod secret-pod
kubectl -n rising delete secret my-secret
```

1.7 TODO Node log `NO1` - Node inspection

- Assess the available nodes in the cluster:

```
kubectl get nodes
```

- Inspect one of the nodes:

```
kubectl describe node kubeadm-worker1
```

1.8 TODO Certificate log `CSR1` - Create a CertificateSigningRequest and approve it

- Note: an helper script is available in `scripts/generate_k8s_user.sh` that automates what is done in this section.

- Use openssl to generate a key and a certificate signing request:

```
export TEMPDIR=$(mktemp -d)
export ORIGINDIR=$(pwd)
cd $TEMPDIR
export KUBEUSER=malicious-user
openssl genpkey -out ${KUBEUSER}.key -algorithm Ed25519
openssl req -new -key ${KUBEUSER}.key -out ${KUBEUSER}.csr -subj
↪  "/CN=${KUBEUSER}/O=rising/O=fbk/"
echo "Key created, available in ${TEMPDIR} for user ${KUBEUSER}"
```

- Create a CertificateSigningRequest:

```
cat <<EOF | kubectl apply -f -
    apiVersion: certificates.k8s.io/v1
    kind: CertificateSigningRequest
    metadata:
      name: ${KUBEUSER}
    spec:
      request: $(cat ${KUBEUSER}.csr | base64 | tr -d '\n')
      signerName: kubernetes.io/kube-apiserver-client
      usages:
      - client auth
    EOF
```

- Deny the CertificateSigningRequest:

```
kubectl certificate deny ${KUBEUSER}
```

- Delete the failed CSR and generate a new keypair for a legitimate user:

```
rm ${KUBEUSER}.csr ${KUBEUSER}.key
kubectl delete csr ${KUBEUSER}
export KUBEUSER=rising-user
openssl genpkey -out ${KUBEUSER}.key -algorithm Ed25519
openssl req -new -key ${KUBEUSER}.key -out ${KUBEUSER}.csr -subj
↪  "/CN=${KUBEUSER}/O=rising/O=fbk/"
echo "Key created, available in ${TEMPDIR} for user ${KUBEUSER}"
```

- Generate a new CertificateSigningRequest:

```
cat <<EOF | kubectl apply -f -
    apiVersion: certificates.k8s.io/v1
    kind: CertificateSigningRequest
    metadata:
      name: ${KUBEUSER}
    spec:
      request: $(cat ${KUBEUSER}.csr | base64 | tr -d '\n')
      signerName: kubernetes.io/kube-apiserver-client
      usages:
      - client auth
    EOF
```

- Inspect the status of the CSR:

```
kubectl get csr ${KUBEUSER} -o yaml
```

- Approve the CertificateSigningRequest:

```
kubectl certificate approve ${KUBEUSER}
```

- Obtain the certificate from the CSR and store it in the folder

```
kubectl get csr ${KUBEUSER} -o jsonpath='{.status.certificate}' | base64 -d > ${KUBEUSER}.crt
head -n 2 ${KUBEUSER}.crt
```

- Use the certificate to generate a `.kubeconfig` for the user:

```
cat <<EOF > ${KUBEUSER}.kubeconfig
    apiVersion: v1
    kind: Config
    preferences: {}
    clusters:
    $(kubectl config view --raw --minify --flatten | yq .clusters)
    contexts: []
    users: []
    EOF
kubectl --kubeconfig ${KUBEUSER}.kubeconfig config set-credentials ${KUBEUSER}
↪  --client-certificate=${KUBEUSER}.crt --client-key=${KUBEUSER}.key --embed-certs=true
kubectl --kubeconfig ${KUBEUSER}.kubeconfig config set-context ${KUBEUSER}
↪  --cluster=kubernetes --user=${KUBEUSER} --cluster=risinglab
kubectl --kubeconfig ${KUBEUSER}.kubeconfig config use-context ${KUBEUSER}
cp ${KUBEUSER}.kubeconfig ${ORIGINDIR}
```

1.9 TODO Role log `R1` - Create a Role and bind it to a User

- **Prerequisite**: complete `CSR1` and obtain the credentials for the user. Make sure they are available as `rising-user.kubeconfig` in the current directory.

- Create a Role:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: Role
    metadata:
      name: rising-reader
      namespace: rising
    rules:
    - apiGroups: [""]
      resources: ["pods"]
      verbs: ["get", "list"]
    EOF
```

- List the roles in the namespace:

```
kubectl -n rising get roles
```

- We changed idea: let's also give `get`, `list`, `patch` and `watch` for Secrets:

```
kubectl patch role rising-reader -n rising --type='json' -p='[{"op": "add", "path":
↪  "/rules/0", "value": {"apiGroups": [""], "resources": ["secrets"], "verbs": ["get",
↪  "list", "patch", "watch"]}}]'
```

- Let's create a RoleBinding and assign it to the user:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: RoleBinding
    metadata:
      name: pod-reader-binding
      namespace: rising
    subjects:
    - kind: User
      name: rising-user
    roleRef:
      kind: Role
      name: rising-reader
      apiGroup: rbac.authorization.k8s.io
    EOF
```

- Test the permissions of the user:

```
kubectl --kubeconfig rising-user.kubeconfig get pods -n rising
kubectl --kubeconfig rising-user.kubeconfig get secrets -n rising
kubectl --kubeconfig rising-user.kubeconfig delete pod non-existent-pod -n rising
```

1.10 Role log `R2` - Create a ClusterRole and bind it to a Group

1.11 Role log `R3` - Create a ServiceAccount for a long-running Pod and bind it to a Role

1.12 Access Review `AR1` - Use can-i to view permissions of self, other users, over different namespaces

1.13 Access Review `AR2` - Use auth whoami to view the current user